Can you spot a future top-cited AI researcher in their first 3 years?

I followed 7,159 AI researchers for 15 years using open data and asked what their first three years reveal.

Swipe for the method, the code and the results →

Association, not cause. Preliminary results from one field.

Two questions

- Which signals in the first 3 years of an AI career predict becoming a top-5% cited researcher by year 15?
- Does working with a top researcher early matter more for people at lower-ranked institutions?

"Top 5%" here means at least 1,540 citations, counting every paper published in the researcher's first 15 years. 358 of 7,159 researchers (5.0%) reach it.

I report whatever the data shows, including the parts that did not come out.

Who is in the study

- 40,000 candidate authors, found on a random sample of AI papers from 2008-2011
- 12,610 of them published their first indexed paper in 2008-2011
- 7,159 published at least 2 papers in their first 3 years: this is the cohort

People who later stopped publishing stay in the data with the label "not top 5%". 30% of the cohort published nothing after year 3. Dropping them would make careers look better than they were (survivorship bias).

Source: OpenAlex, a free open catalogue of research papers (CC0).

What the model sees

Only information from the first 3 years (no peeking at the future):

- Papers in first 3 years
- Different venues
- Papers in top venues
- Worked with a top researcher
- Different coauthors
- Average team size
- Institution rank group
- Topic variety
- Share as first author
- Start year
- Citations to the early papers (also tested without)

Train on researchers who started in 2008-09 (n=4,210). Test on those who started in 2010-11 (n=2,949): the model never sees the test group.

The code: cohort and outcome

```python
# src/cohort.py
def select_cohort(stage_a: pd.DataFrame) -> tuple[pd.DataFrame, dict]:
    log = {"candidates_with_works": int(stage_a.author_id.nunique())}
    d = stage_a.dropna(subset=["year"]).drop_duplicates(["author_id", "work_id"])
    t0 = d.groupby("author_id").year.min().rename("t0").reset_index()
    log["t0_in_range"] = int(t0.t0.between(*config.T0_RANGE).sum())
    t0 = t0[t0.t0.between(*config.T0_RANGE)]
    m = d.merge(t0, on="author_id")
    early = m[(m.year >= m.t0) & (m.year <= m.t0 + config.EARLY_WINDOW - 1)]
    n_early =
        early.groupby("author_id").work_id.nunique().rename("n_early").reset_index()
    t0 = t0.merge(n_early, on="author_id", how="left").fillna({"n_early": 0})
    t0 = t0[t0.n_early >= config.MIN_EARLY_WORKS].reset_index(drop=True)
    log["cohort_after_min_early_works"] = int(len(t0))
    return t0, log

# src/target.py
def build_targets(outcome: pd.DataFrame, cohort: pd.DataFrame) -> pd.DataFrame:
    g = outcome.groupby("author_id").cites
    agg = pd.DataFrame({"citations_15": g.sum(), "n_papers_15": g.size(),
                        "h_index_15": g.apply(h_index)})
    t = cohort[["author_id", "t0"]].merge(agg, on="author_id", how="left")
    t[["citations_15", "n_papers_15", "h_index_15"]] = t[["citations_15",
        "n_papers_15", "h_index_15"]].fillna(0).astype(int)
    thr = float(np.quantile(t.citations_15, 1 - config.TOP_FRAC))
    t["label"] = (t.citations_15 >= thr).astype(int)
    t.attrs["top5_threshold"] = thr
    return t
```

The code: model and explanation

```python
# src/modeling.py
def temporal_split(df: pd.DataFrame):
    tr = df[df.cohort_year.between(*TRAIN_YEARS)]
    te = df[df.cohort_year.between(*TEST_YEARS)]
    return tr, te


def make_lgbm(pos_rate: float):
    return LGBMClassifier(n_estimators=300, learning_rate=0.03, num_leaves=15,
        min_child_samples=20,
                          subsample=0.8, subsample_freq=1, colsample_bytree=0.8,
                          scale_pos_weight=(1 - pos_rate) / max(pos_rate, 1e-6),
                          random_state=config.SEED, verbose=-1)
```

```python
# src/explain.py
def shap_explain(model, X: pd.DataFrame, interactions: bool = True):
    ex = shap.TreeExplainer(model)
    sv = _pos_class(ex.shap_values(X))
    iv = None
    if interactions:
        iv = ex.shap_interaction_values(X)
        iv = iv[1] if isinstance(iv, list) else iv
    return sv, iv
```

```python
# src/explain.py
def group_assignment(df: pd.DataFrame) -> pd.Series:
    g = pd.Series(np.nan, index=df.index, dtype=object)
    g[df.institution_tier == 1] = "Top-ranked institution"
    g[df.institution_tier.isin([3, 4])] = "Lower-ranked institution"
    return g
```

Result 1: the first 3 years do carry a signal

0.83

AUC on researchers the model never saw (95% range 0.79-0.87). 0.50 would be a coin flip, 1.00 perfect.

AUC asks: pick one future top-5% researcher and one other at random; how often does the model score the top-5% one higher?

But the model is far from a crystal ball: precision-recall AUC is 0.31 against 0.04 for guessing. Most top-5% researchers are not identifiable at year 3. A plain logistic regression does about as well (AUC 0.87).

How well can the first 3 years predict the top 5%?

LightGBM, trained on 2008-09 starters, tested on 2010-11 starters

Ranking accuracy (AUC)

0.83

95% range 0.79-0.87 (0.50 = coin flip)

Precision-recall (PR-AUC)

0.31

95% range 0.23-0.40 (chance = 0.04)

Researchers in study

7,159

2,949 in the held-out test group

Top-5% share in test group

4.2%

the base rate to beat

Data: OpenAlex (CC0)

Reading the SHAP chart

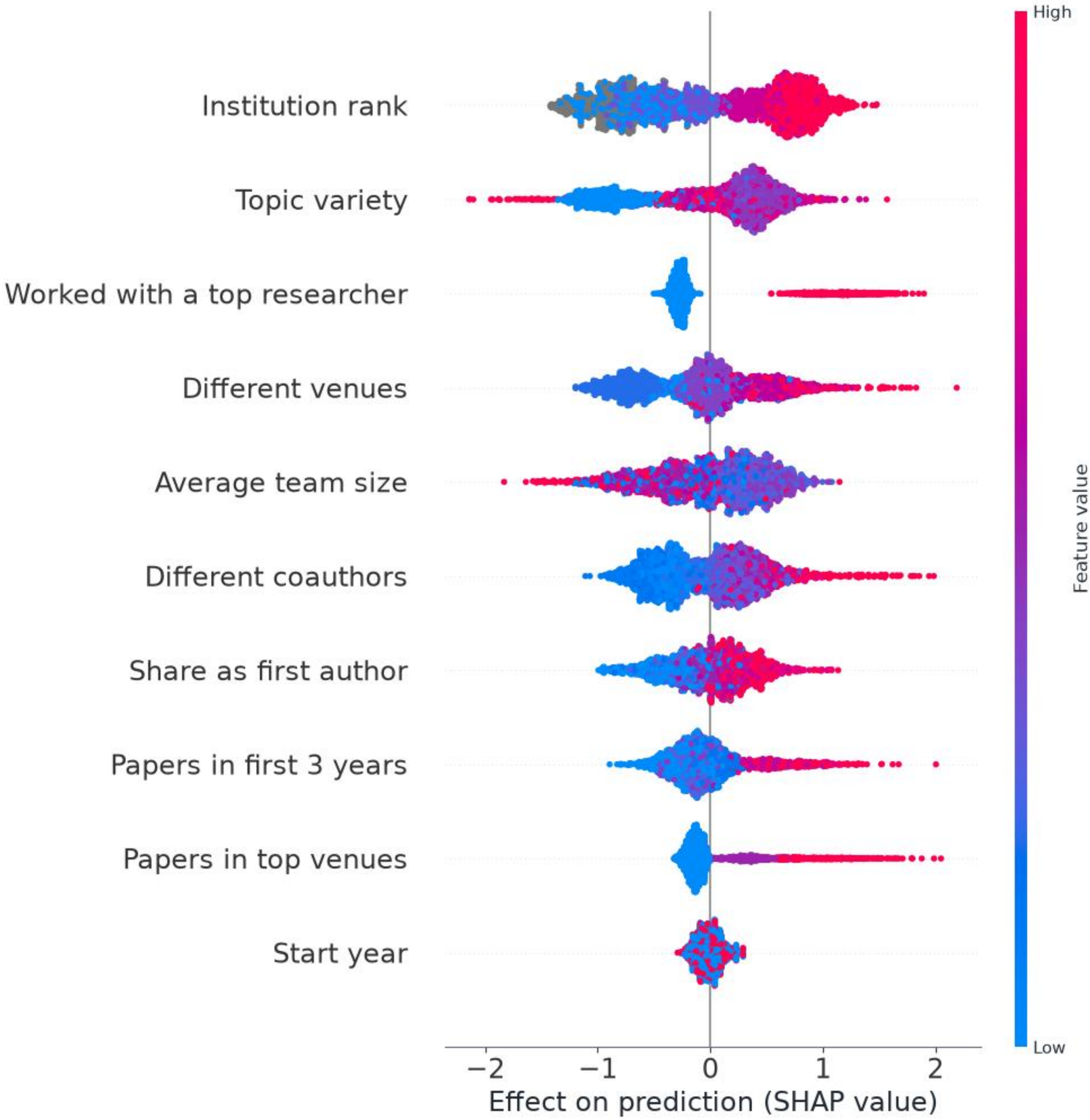
SHAP splits each prediction into contributions, one per signal.

- Each dot is one researcher.
- Right of zero: that signal pushed the prediction toward top 5%. Left: away from it.
- Red = a high value of the signal, blue = low. Grey = unknown.
- Signals are sorted by overall influence, most influential on top.

Explains the model, not the world: the model uses whatever statistically goes with the outcome.

What do the first three years say about a researcher's future?

Each dot is one researcher. Right = pushes toward 'top 5% by year 15'.



Red = high value of the signal, blue = low. Institution rank: red = top-ranked, grey = unknown.

Data: OpenAlex (CC0)

Result 2: a top coauthor goes with better odds

No model needed: just count who reached the top 5%.

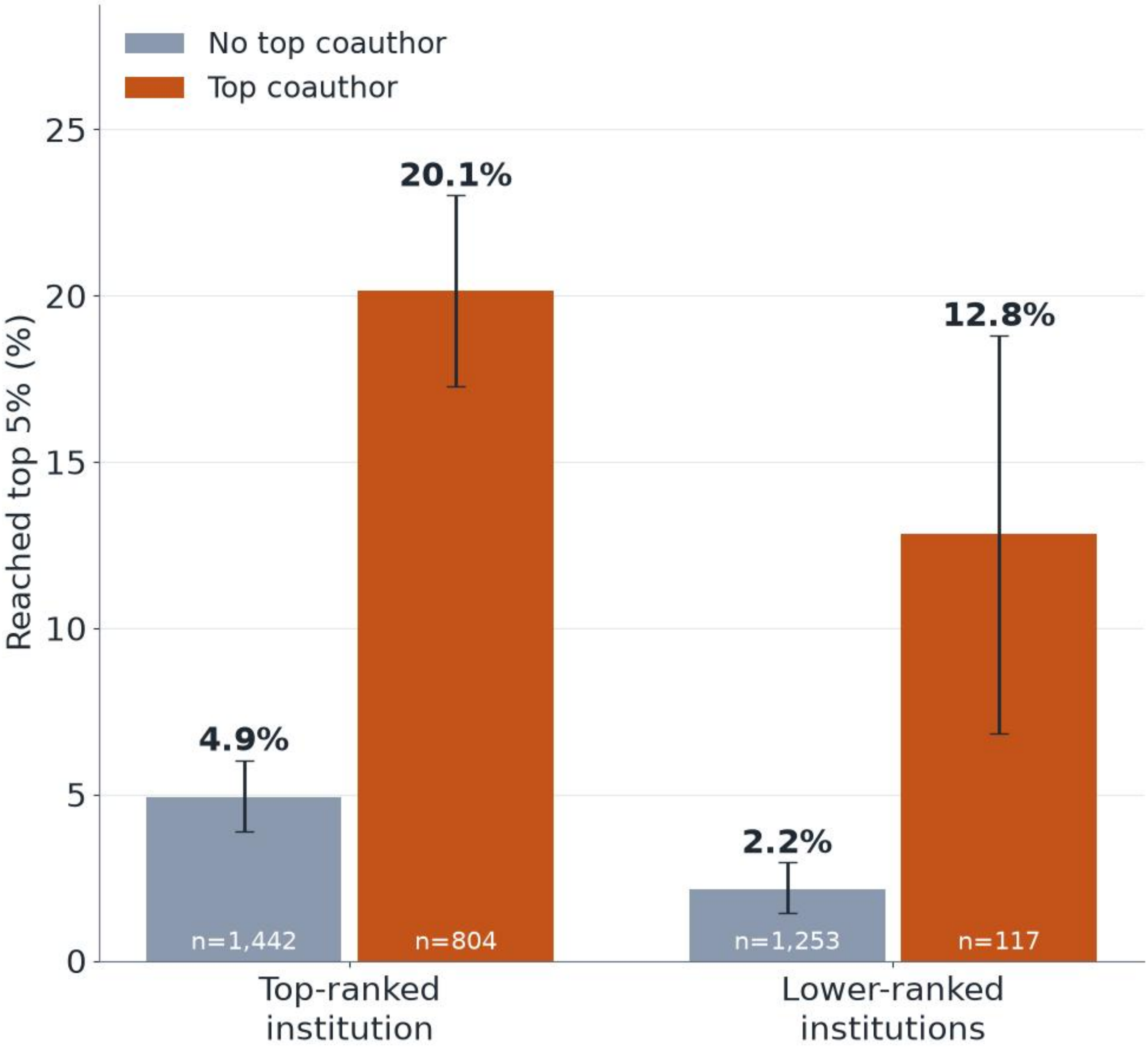
- Top-ranked institution: 4.9% without a top coauthor vs 20.1% with one
- Lower-ranked institution: 2.2% vs 12.8%

Overall gap: 15.6 percentage points (95% range 12.9 to 18.1). It survives adjusting for the other signals (log-odds 0.44, range 0.34 to 0.55), and its direction matches Li et al. 2019 (Nature Communications).

"Top coauthor" = an early coauthor in the top 5% by current h-index. Papers written together also add to the researcher's own citations, which builds part of the link in.

Early work with a top researcher goes with better odds of top-5% citations

Share reaching the top 5% most-cited by year 15 (association only)



Bars show 95% confidence ranges. Top-ranked = top quarter of institutions; lower-ranked = bottom half. Excludes 3,543 researchers with a middle-ranked or unknown institution.

Data: OpenAlex (CC0)

Result 3: does it matter MORE at lower-ranked institutions?

Can't tell

The gap is 15.2 points at top-ranked and 10.7 at lower-ranked institutions. Difference: -4.6 points (95% range -10.9 to +2.6).

The range includes zero, so the data do not support the idea that a top coauthor matters more for people from lower-ranked institutions, and they do not rule it out either.

Only 117 lower-ranked researchers had a top coauthor (15 of them reached the top 5%). That is too few to separate a real difference from noise.

What this does not show

- Correlation, not causation: strong researchers choose strong groups and vice versa.
- One field (AI) and one database. No claim beyond that.
- Many conferences (ICML, ICLR, CVPR, ICCV, ...) are not matched in OpenAlex, so venue quality is understated.
- 33% of researchers have no institution; the 2x2 leaves out 3,543.
- Author-name mix-ups in OpenAlex can merge or split careers.

Reproduce it: repo xai-sketchbook, folder sketches/01-early-career-signals

```
python -m src.pipeline  |  python -m src.carousel
```